

Homepage map: how to load the catalog

This note is the data-loading decision for the Destination Discovery Map. It answers: **should the frontend refetch from the backend on every zoom and preference tap, or load the homepage slice once?**

Recommendation for v1: load the homepage-map slice in one go. Keep it in memory. Preference taps and country → city → POI stay local. Google tiles (or the custom basemap) remain the slow piece — not our JSON.

Related: [client-requirements.md](#) (the brief), [API.md](#) (endpoint contract), [Homepage_MAP_API.md](#) (architecture).

Today the prototype already works this way: `src/homepage-map/data/catalog.ts` is in memory and there is no `fetch()`. The map is instant because of that, not because Google is fast.

The question

When a backend exists, there are two instincts:

- 1. **Fetch per level** — world, then Poland’s cities, then Kraków’s attractions, then Wawel’s products. Refetch again when the traveller taps a preference chip.
- 2. **Fetch once** — one payload with every pin, list row, and preference **fits** the homepage is allowed to show. The client re-ranks and flies the camera.

Instinct 1 matches a typical REST map API. Instinct 2 matches this product. The brief is a **recommendation funnel on a small, capped catalog**, not a “show everything” explorer.

What the brief actually requires

From *Destination Discovery Map — Client Requirements*:

Constraint	Where	What it means for data
Preference tap re-ranks everything with one tap	§8 Preference row	A chip must not wait on the network
Transition line states what changed, never “loading”	§11 Navigation	Drill-in must not stall on a round-trip
Preference is session-only, never a URL	§13 Indexability	You cannot cache a separate page per audience
The list is the content ; the map is an enhancement	Principle 7	SSR / crawlable list still matters; JSON is for the map
Commercial detail arrives late	Principle 5	Prices from L2; full facts at L3 — live prices can wait

Constraint	Where	What it means for data
Caps: ≤8 cities, ≤4 attractions, ≤3 products	Principle 4	The homepage payload is small by design
Not a “show everything” catalogue view	What this map is NOT	UI rule: do not pin the whole company catalog. Not a network rule.
Poster / first paint — map never blocks first render	§12 States	Catalog fetch can happen behind a poster; do not block HTML
Pan never changes level	§11 Movement	Never fetch on drag
Loading level skeleton is a designed state	§12	Allowed on first paint / rare misses — not on every pin tap

The current endpoint walkthrough in [API.md](#) §9 (one request per drill-in and per preference) **conflicts with §8 and §11**. That is why this document exists.

What “all data” means

The homepage-map slice only. Not every tour RosoTravel sells.

Entity	Cap on this surface	In the current catalog
Countries	All covered destinations	6 (pl , it , jp , pt , gr , th)
Cities per country	≤ 8 (readability may show fewer)	~5 extra cities + 1 featured city per country
Attractions per city	≤ 4 winners	4 per featured city; extras are thinner
Products per attraction	3 (4 if readable)	2–3
Audiences	5 chips	first , family , culture , active , budget

Include **[fits](#)** (**tier, chips, why, trade-off**) for every audience in that first payload. Preference is not in the URL, so the client must be able to re-rank without another GET.

That JSON is tens of kilobytes. Google map tiles dwarf it. Do **not** download the company-wide booking catalogue “just in case”.

```

First paint
  → one bootstrap payload (homepage slice + all audience fits)
    → in-memory catalog
      → preference tap → re-rank locally → flash “Re-ranked for ...”
      → country / city / POI → swap pins from memory → fly camera

```

Approaches

Each approach: how it works, when the user waits, fit to the brief, verdict.

A — Full-slice one-go (best for v1)

How. One `GET /bootstrap` (or a static JSON file) returns:

- all destinations, with `fits` for all five audiences
- cities per country (lat/lng, picks)
- attractions + products for those cities
- audience badge counts

The client keeps this in memory and ranks locally — the same model as `catalog.ts` + `ranking.ts` today.

Moment	Wait
First paint	One catalog request, behind the poster
Drill-in (Poland → Kraków → Wawel)	None
Preference chip	None
Pan / breadcrumb / zoom out	None

Fits the brief. One-tap re-rank. Flash states what changed, never “loading”. Caps keep the payload honest.

Verdict: do this for v1.

B — One-go, already ranked for one audience

How. Same full tree as A, but the backend returns copy already ranked for `?audience=family`. Tapping Culture fetches the whole tree again.

Moment	Wait
First paint	One request
Drill-in	None
Preference chip	One full-tree refetch

Fits the brief. Drill-in stays snappy. Preference tap does not: it fights “re-ranks with one tap” and “never loading”.

Verdict: acceptable only if ranking must stay server-only. Prefer A. If you must use B, keep the previous audience’s pins on screen until the new payload arrives — never blank the map.

C — Per-level fetch (current API.md §9)

How. World, then `GET .../countries/pl/cities`, then attractions, then products. Preference tap refetches the current level.

Tap Poland
→ wait 100–400ms for cities
→ then fly the camera

Moment	Wait
First paint	Bootstrap (world only)
Each drill-in	A new round-trip before the camera
Each preference tap	Another round-trip
Four levels + one chip	Five network waits on a phone

Fits the brief. Poor. The flash cannot honestly skip “loading”. Principle 7 still needs a crawlable list; this pattern does not help SEO, it only slows the map.

Verdict: not for v1. Keep the granular URLs for other clients or a later live-price layer. Do not make them the homepage default.

D — Per-level fetch + in-memory cache

How. Same as C, but the first visit to Poland is cached. Opening Poland again is instant.

Fits the brief. Better than C. The **first** tap on each country/city still stalls the funnel — the tap that matters most.

Verdict: better than C, still first-tap lag. Not worth the complexity when the whole slice is tiny.

E — SSR list HTML + one JSON for the map (best long-term, with A)

How. The ranked list below the map is **server-rendered HTML** (crawlable destinations, real links). The map hydrates from the **same** homepage-slice payload (inline JSON or the A bootstrap).

This is principle 7: *the map is an enhancement, not the content*. If Google (or the custom canvas) fails, the list is still the section.

Fits the brief. Best match for indexability + fallback. Preference still must not change the URL; the HTML list is a default ranking (e.g. family), and the client reorders it in the session.

Verdict: do A now in this Vite prototype; add E when the real homepage (likely Next.js) exists.

F — Static JSON on a CDN

How. Ship today’s `catalog.ts` as `/homepage-map.json`. No custom API. Cache at the edge.

Fits the brief. Same runtime behaviour as A. Fine until a CMS or live prices exist.

Verdict: fine as the first backend. Graduate to a real `GET /bootstrap` when editors need to change copy without a deploy.

G — Hybrid: one-go pins + live L3 commercial data

How. A/F payload has everything needed to pin and explain. When the traveller reaches an attraction, a second request loads **live** price, availability, or review slices.

Matches principle 5: commercial detail arrives late. The camera has already moved; the product cards can skeleton for a moment.

Fits the brief. Right **later**, when L3 numbers must be live. Wrong as the first architecture — today's catalog prices are static.

Verdict: use when booking facts go live. Not v1.

Comparison

	First paint	Drill-in	Preference tap	Brief fit	Use when
A — Full-slice one-go	1 request	Instant	Instant	Strong	v1 default
B — One-go, ranked per audience	1 request	Instant	Refetch	Medium	Ranking cannot leave the server
C — Per-level fetch	1 request	Wait each level	Wait	Weak	Do not use on the homepage
D — Per-level + cache	1 request	Wait first visit	Wait or cache	Weak	Do not use when A is possible
E — SSR list + A JSON	HTML + JSON	Instant	Instant	Strongest	Production homepage
F — Static CDN JSON	1 file	Instant	Instant	Strong	No CMS yet
G — Hybrid live L3	1 request + later L3	Instant, then maybe L3	Instant	Strong later	Live prices / availability

Recommendation

1. **Now (this prototype, and the first API): Approach A.** One bootstrap of the homepage slice, including **fits** for every audience. Frontend: load once, cache, fly camera locally. Preference chip = no network.
2. **When the homepage is real SSR: A + E.** Crawlable list in the page; map reads the same slice. Preference still never lands in the URL.
3. **When L3 prices or availability must be live: G on top of A.** Do not go back to C for pins and copy.
4. **Until then: F** (static JSON) is a valid A without a custom backend.

Keep the granular endpoints in [API.md](#) for debugging, other surfaces, or a future G call. They are **not** the v1 navigation path.

What not to do

- **Fetch on pan.** Pan never changes level. No catalog request, ever.
- **Fetch the company-wide catalogue.** Caps exist so this surface stays a shortlist. A 2 MB "everything" payload is the wrong reading of "one go".
- **Put preference in the URL** so you can cache [/family](#) vs [/culture](#). The brief forbids it. Indexable structure stays stable; ranking is a session signal.
- **Block first paint** on Google + catalog. Show the poster (and the SSR list, when it exists). Hydrate the map behind it.
- **Show "Loading..."** on the flash line. The line is "Poland · 5 cities prioritised", not a spinner. A skeleton is allowed on **first** paint / error recovery, not on every drill-in.
- **Refetch on breadcrumb or zoom-out** when the slice is already in memory.
- **Rank only on the server and omit [fits](#).** Then every chip tap is Approach B or C. If ranking stays server-side, send the full tree per audience (B), not one level (C).
- **Wait for the network, then fly the camera.** Fly from local data. If a later G request is in flight, keep last pins and fill commercial fields when they arrive.

How this maps to the API contract

Granular reads stay documented. **Bootstrap changes meaning.**

Endpoint	Role in v1
GET /bootstrap	Default. Full homepage slice: audiences, destinations, cities, attractions, products, fits for all audiences. Optional ?audience= only to rank the first paint / matchline.
GET /destinations	Optional. Other clients, or debug. Not needed after bootstrap.
GET /countries/{id}/cities	Optional after bootstrap.
GET /cities/{id}/attractions	Optional after bootstrap.
GET /cities/{id}/attractions/{id}	Optional; later the G hook for live commercial data.

Frontend session:

1. Mount → [GET /bootstrap](#) once (or read static JSON).
2. Cache the tree.
3. [goCountry](#) / [goCity](#) / [goPoi](#) / [setAud](#) read memory, set [level](#), fly camera, set the flash.
4. On error: keep last good state; degrade to the list (principle 7). Do not blank the map.

Worked flow after A (no extra GETs):

1. GET /bootstrap
→ preference bar + 6 countries + the rest of the tree in memory
2. Tap Poland
→ cities from memory, camera zoom ~6.2
→ flash: "Poland · 5 cities prioritised"
3. Tap Kraków
→ attractions from memory
→ flash: "Kraków · 4 attractions prioritised"
4. Tap Wawel Castle
→ products from memory
→ flash: "Wawel Castle · 2 products from €32"
5. Tap Budget-smart
→ re-rank from `fits` in memory
→ flash: "Re-ranked for budget-smart"
→ if Poland is a poor fit (tier > 2), return to world – still no fetch

Rough size

The current local catalog is the right order of magnitude for production v1:

- 6 countries × 5 audience **fits**
- on the order of 30 city records
- up to 4 attractions × a few products

That is **tens of KB of JSON**, gzipped smaller. One extra HTTP handshake per pin tap costs more *felt* time than sending the unused cities up front.

The expensive bytes on this page are **map tiles / 3D tiles**, not destinations.

Recommended line for the client

The homepage map is a short, capped recommendation tree, not a live search of the whole catalogue. We load that slice once so preference and zoom stay instant — the brief says we re-rank with one tap and never flash "loading". The list under the map stays the crawlable content. We only fetch again later if booking prices must be live at the last step.